



Algoritmos y Estructuras de Datos

1º Año
Ingeniería en Sistemas de Información

Arreglos

ARREGLOS: Tipo de Datos Estructurados

Hasta el momento se han estudiado los datos de tipo simple cuya característica común es que cada variable representa a un solo dato individual.

La diferencia con los datos estructurados es que un único identificador puede representar a múltiples datos individuales, siendo referenciados en forma separada cada uno de éstos por medio de índices.

ARREGLOS UNIDIMENSIONALES

Python no maneja la estructura de arreglos tal como la vamos a ver en este curso.

La estructura de datos que se asemeja en Python a un arreglo es la estructura de Lista.

La diferencia entre un arreglo y una lista es que un arreglo es una estructura estática(fija), mientras que una lista es una estructura dinámica, es decir que puede crecer y decrecer. Otra característica que las diferencia es que un arreglo no permite mezclar tipos de datos mientras que una lista sí lo permite.



Un arreglo entonces puede ser considerado como un conjunto de datos, todos del mismo tipo, identificados mediante un nombre de variable. Un arreglo es una variable compuesta en vez de ser una variable simple como trabajábamos hasta ahora.

Cada elemento (cada dato) se lo referencia con el nombre del arreglo seguido de un índice (también llamado subíndice) encerrado entre corchetes.

Variable nombre del arreglo: ARREGLO [0..cantidad de posiciones] de tipo de dato de los elementos que contendrá el arreglo

El tipo de índice puede ser de tipo simple: entero, char, **nunca real. (Python solo nos permite un índice entero)**

El tipo de elementos puede ser cualquiera: entero, char, booleano, float, string, registro.

Al ser una estructura de datos **estática** su dimensión debe estar fijada de antemano. Es decir, debemos saber qué cantidad de elementos vamos a poder guardar en un arreglo.

Un arreglo es una variable compuesta!

Le tengo que dar un **nombre**

Debe estar **dimensionado**

Y le tengo que decir qué **tipo de datos** podrá almacenar.

Variable



Los elementos del arreglo pueden ser utilizados en estructuras de control, comparación, sentencias de asignación, sentencias de lectura y escritura, etc., identificándolos con el valor del índice correspondiente, el cual puede ser una constante, una variable o una expresión.

CARGA y EXHIBICION DE ARREGLOS Unidimensionales

```
#Constante
Tam= 10
Variable
V: arreglo [0 ..Tam ] de tipo de dato
```

En Python para definir un arreglo de 10 elementos de tipo entero lo hacemos de la siguiente forma:

```
Tam=10
V=Tam*[0]

def Cargar(V,Tam):
    for i in range(0,Tam):
        V[i]=int(input("Nro: "))

Def Mostrar(V,Tam):
    for i in range(Tam): #muestro
        print("posición ",i," ",V[i])
```

Notar que $V=Tam[0]$ equivale en python a realizar el procedimiento Limpiar*

Def Limpiar(V,Tam)

For I = 0 a Tam

$V[I] = 0$

Def Cargar(V,Tam)

For I = 0 a Tam

Leer ($V[I]$)

Def Mostar(V,Tam)

For I = 0 a Tam

mostrar ($V[I]$)

Al tener un arreglo cargado en memoria, nos permite hacer consultas sobre la información que el mismo contiene. Para ello se pueden hacer búsquedas para encontrar un dato concreto. Los métodos de búsquedas son 2: **Secuencial o Dicotómica**.

La búsqueda secuencial comienza desde la 1era. posición hasta que lo encuentra, o hasta el final del arreglo.

Para realizar una búsqueda secuencial, no es necesario tener el arreglo ordenado.

Igualmente, si está ordenado o desordenado, la cantidad de veces que procesará este módulo para encontrar un dato, como máximo será igual a la longitud -límite- del arreglo.

Las búsquedas secuenciales, si bien son más lentas, son útiles para buscar información, la cual se encuentre repetida dentro de una estructura.

Si se sabe que el elemento a buscar solo aparece una única vez o no está, la estructura de control a utilizar sería alguna estructura de repetición condicionada para que salga del recorrido cuando encuentre el elemento.

Si el elemento a buscar se repite varias veces dentro del arreglo y queremos saber cuántas veces aparece podríamos utilizar una estructura de For ya que deberemos recorrer el arreglo completo.

BÚSQUEDA SECUENCIAL EN ARREGLOS UNIDIMENSIONALES

Ejemplo:

Ingresa 30 elementos enteros **distintos** en un arreglo. Ingresa luego un valor entero. Se desea determinar si dicho valor coincide con algún elemento del arreglo; si es así, exhibir el valor y la posición que ocupa en el arreglo, sino, informar dicha situación

Programa Principal

CARGA(V)
Leer (N)
BUSCAR(V,N)

Def BUSCAR(X:vec,e:entero)

Def BUSCAR(x,e)

i = - 1	
i = i + 1	
Hasta que (X[i]=e) or (i=30)	
X[i]=e	
v	f
Mostrar ("el elemento", e, "se encuentra en la posición", i)	Mostrar ("No se encontró")

En Python podemos hacerlo usando mientras y también utilizando una función en vez de un procedimiento.

Programa Principal

CARGA(V)	
Leer (N)	
Posicion = FUNBUSCAR(V,N)	
Posición != -1	
v	f
Mostrar (N, Posicion)	Mostrar ('No se encontró')

Def FUNBUSCAR(X:vec ; e:entero) : entero

i = 0	
mientras (X[i] != e) AND (i<29)	
i = i + 1	
X[i] = e	
v	f
Return i	Return - 1

```
def buscar(X,e):
    i = 0
    while (X[i] != e and i < 29):
        i = i + 1
    if X[i]==e:
        return i
    else:
        return -1
```

ORDENAMIENTO en Unidimensionales

El mismo se demuestra mediante el método del Falso Burbuja.

Dicho método es el más sencillo de comprender dentro del ámbito académico, pero no se utiliza a nivel profesional dado que es el más lento de todos los existentes.

La lentitud se ve reflejada en el sistema de comparaciones, donde el 1er. elemento del arreglo compara con todos los restantes, desde el 2do. en adelante, provocando intercambios si se cumple la condición de ordenar dentro del IF, quedando como resultado el menor de todos en la 1era. posición.

Luego repite el mismo mecanismo comenzando con el 2do. elemento y comparándolo desde el 3ero en adelante. Así sucesivamente hasta comparar el penúltimo con el último.

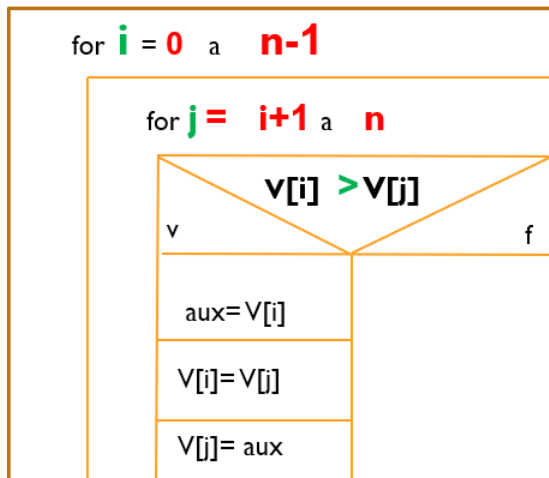
Notar que para ordenar creciente o decreciente solo será necesario cambiar el signo de comparación mayor o menor.

Ejemplo: Ordenar un arreglo unidimensional de menor a mayor (Falso Burbuja):

(* intercambiar de menor a mayor los elementos de un array unidimensional *)

Def ORDENO(V)

Procedimiento ORDENO(var V:vec)

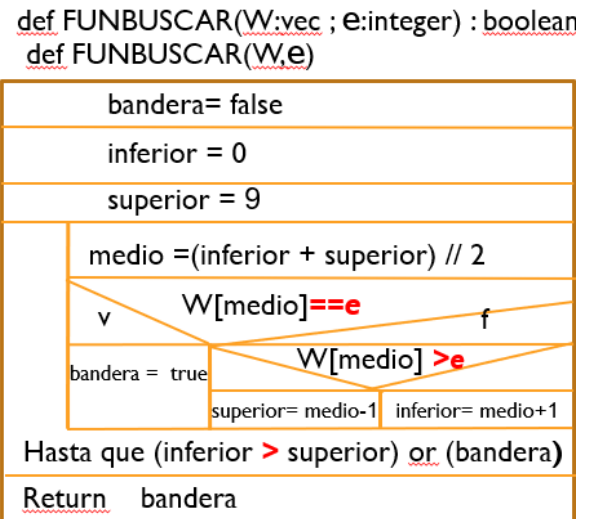
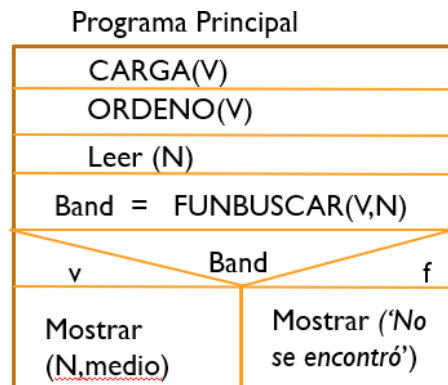


```
def ordeno(V):
    for i in range(n-1):
        for j in range(i+1,n):
            if(V[i]>V[j]):
                aux = V[i]
                V[i] = V[j]
                V[j] = aux
```

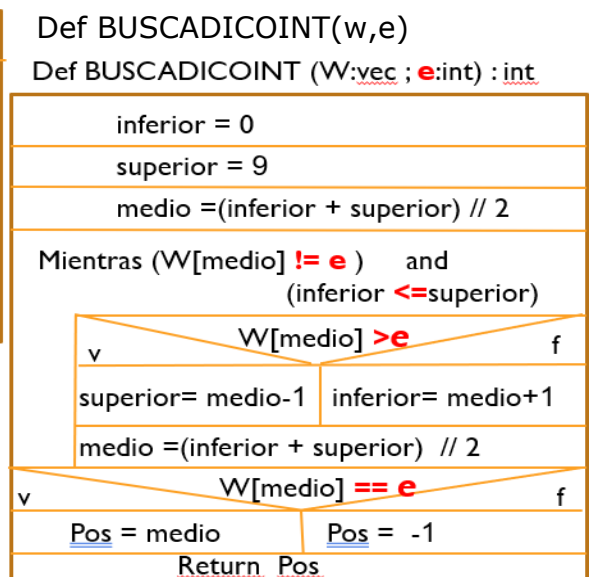
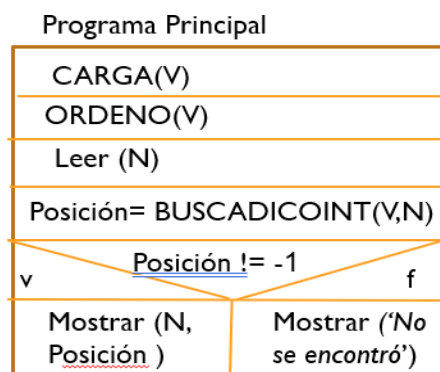
BUSQUEDA DICOTOMICA en Unidimensionales

En el método dicotómico o Binario, en primera instancia se debe verificar el ordenamiento del arreglo, y ordenarlo en caso que sea necesario, para luego buscar.
 La búsqueda dicotómica se posiciona en la mitad del arreglo para hacer la búsqueda.
 En caso de coincidir finaliza el proceso, de lo contrario seguirá buscando, pero en la mitad del arreglo en la cual posiblemente encuentre el dato (por estar ordenado).
 Para ello verificará si busca desde el principio hasta la mitad menos uno (porque en la mitad no estaba el dato) ó si lo busca desde la mitad más uno hasta el final.
 La cantidad de veces que se repetirá el proceso será como máximo, la mitad de la longitud del arreglo + 1 pasadamás. Por ejemplo, si quisiéramos recorrer un arreglo de 10 posiciones:

Usando repetir-hasta y una función booleana



Usando mientras como estructura de repetición y una función que retorne un entero con la posición del medio si lo encontró, o el valor -1 si no está.



```

def buscaDicoint(W,e): # función que devuelva la posición ó -1 si no lo encuentra
    inferior = 0
    superior = 9 # vean acá ahora en python el superior será uno menos
    medio = (inf+sup)//2
    while (W[medio]!=e) and (inferior<superior):
        if(e>W[medio]):
            inferior = medio + 1
        else:
            superior = medio - 1
        medio = (inferior+superior)//2
    if (W[medio] == e):
        pos=medio
        #print("El elemento buscado está en la posición: ", medio)
    else:
        pos=-1 #si no lo encuentra que la función devuelva -1
    return pos

```

MERGE (intercalación) en Unidimensionales

Supongamos que tenemos dos arreglos V1 y V2, ambos del mismo tipo de elementos y ambos ordenados con el mismo criterio (creciente o decreciente).
No necesariamente con la misma cantidad de elementos.

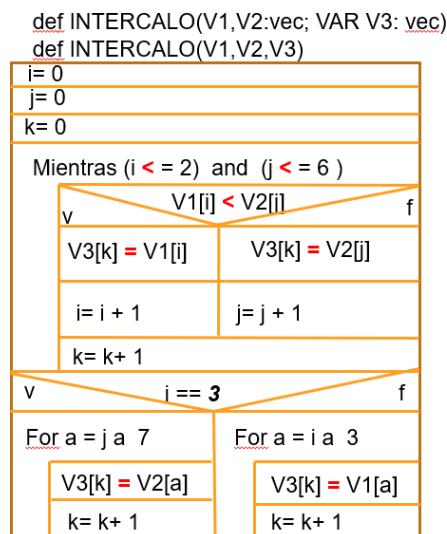
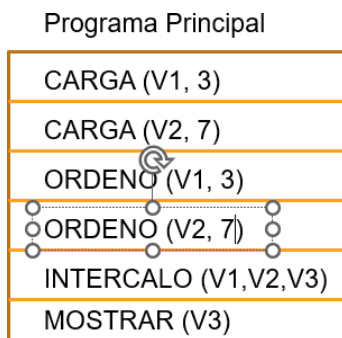
Sea N la cantidad de elementos de V1, y M la cantidad de elementos de V2, queremos armar un nuevo arreglo V3 que contenga todos los elementos de V1 y todos los elementos de V2 (es decir que tenga N+M elementos) quedando ordenado con el mismo criterio.
Los elementos pueden estar repetidos.

Sería ineficiente armar el nuevo arreglo V3, que contenga al principio los elementos de V1, a continuación los de V2, y luego ordenarlo.

Ejemplo: si V1 tiene N = 3 elementos (de 0 a 2)
V2 tiene M = 7 elementos (de 0 a 6), entonces ...
V3 tendrá 7 + 3 = 10 elementos (de 0 a 9)

Siendo: *i* el índice con que recorremos V1
j el índice con que recorremos V2
k el índice con que recorremos V3

La idea es recorrer desde el principio los arreglos V1 y V2. Comenzar con los índices *i*, *j*, *k* en 0, luego, comparar V1[*i*] con V2[*j*] y al elemento menor se lo copia en V3[*k*]
Incrementar en 1 el índice del arreglo que se copió y a *k* de V3
Así hasta llegar al final de uno de los dos arreglos.
Por último, se deben copiar los elementos del arreglo que aún no se terminó de recorrer, "al final" de V3 desde donde quedó *k*.



V1	V2	V3
0 9	0 7	0 7
1 35	1 20	1 9
2 47	2 35	2 20
	3 43	3 35
	4 50	4 35
	5 73	5 43
	6 92	6 47
		7 50
		8 73
		9 92

Comparo V1[*i*] con V2[*j*] y
Copio el menor a V3[*k*]

```
def intercalo(V1,V2,V3,n,m):
i=0
j=0
k=0
while (i <= n-1) and (j <= m-1 ):    #n-1 y m-1 porque arranco con los índices i, j en cero
    if V1[i] < V2[j]:
        V3[k] = V1[i]
        i= i + 1
    else:
        V3[k] = V2[j]
        j= j + 1
    k= k + 1
```

```

#finaliza el mientras, copio lo que quedó sin pasar a V3
    if i == n:
        for a in range (j,m):
            V3[k] = V2[a]
            k= k + 1
    else:
        for a in range (i,n):
            V3[k] = V1[a]
            k= k + 1

#programa Principal
N=3
M=7
V1=[0]*N
V2=[0]*M
V3=[0]*(N+M)
print('Ingrese los elementos del primer arreglo de ',N,' elementos')
cargar(V1,N)
print('Ingrese los elementos del segundo arreglo de ', M, ' elementos')
cargar(V2,M)
print('primer arreglo ordenado')
ordenar(V1,N)
mostrar(V1,N)
print('segundo arreglo ordenado')
ordenar(V2,M)
mostrar(V2,M)
print('Intercalar generando el tercer arreglo ordenado')
intercalar(V1,V2,V3,N,M)
mostrar(V3,N+M)

```

Porqué es más eficiente Aplicar Merge que copiar V1, a continuación V2 y luego ordenar V3?

Porque en el método de ordenamiento (falsa burbuja), se presentan anidadas dos estructuras de repetición FOR. Lo cual implica una cantidad enorme de sentencias en ejecución que aumenta el tiempo de compilación.

Ejemplo

Para ordenar un arreglo de 200 elementos, se trabajará con dos FOR anidados y si bien el FOR interior, en cada repetición, se ejecuta una vez menos, en promedio, las sentencias interiores se ejecutarán unas 20.000 veces.

(200x100 aproximadamente).

ARREGLOS MULTIDIMENSIONALES

Valen las mismas consideraciones de los arreglos unidimensionales, en cuanto a los tipos de datos y tipos de índices (Python solo admite índices de tipo entero, no de tipo char como en otros lenguajes de programación).

Dentro de los arreglos multidimensionales daremos ejemplos de los bidimensionales compuestos por filas y columnas.

La primera dimensión (es decir el primer índice) se refiere al número de fila y la segunda dimensión (segundo índice) al número de columna.

En el caso de tres dimensiones, se agrega un tercer índice comúnmente llamado fondo o profundidad del arreglo.



VARIABLE

nombre del array: ARREGLO (fila,columna,fondo..., índiceN) de tipo de dato que contendrá el arreglo

índice N si fuera de más de 3 dimensiones (no usamos en este curso)

Se lo puede declarar como:

VARIABLE

matriz : ARREGLO [0 .. 2 , 0 .. 3] de enteros

ó

crearnos un tipo de dato para las filas y columnas
tipo de dato

fila = 0 .. 2

columna = 0 ..3

y luego declarar la matriz como una variable...

VARIABLE

matriz: ARREGLO [fila ,columna] de enteros

ó

también podríamos crearnos la matriz como un tipo de dato estructurado en el caso que tuviéramos que trabajar con varias matrices del mismo tipo.

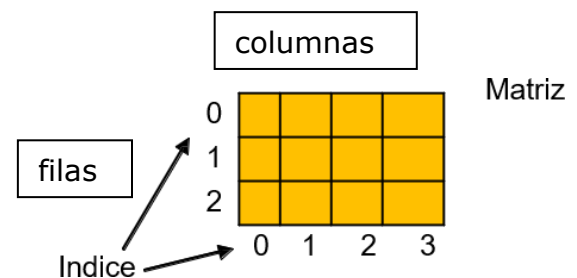
tipo de dato

mat : ARREGLO [0 .. 2 , 0 .. 3] de enteros

y luego declarar las matrices

VARIABLE

M: mat



Notar que para trabajar con una matriz de 3 filas x 4 columnas, en Python comenzamos con el primer índice cero.

También, recordar que en python no necesitamos declarar los tipos de datos acá explicados. Por ello, cuando programamos, aclaramos como comentarios # los tipos de datos utilizados.

CARGA y EXHIBICION DE ARREGLOS Bidimensionales

(supongamos una matriz de 3 filas x 4 columnas)

```
def carga(MM,fil,col):  
    for i in range (fil):  
        print("fila",i)  
        for j in range (col):  
            MM[i][j]= int(input(' Ingrese nro: '))
```

```
Def mostrar(MM,fil,col):  
    for i in range(fil):  
        for j in range (col):  
            print(' ',MM[i][j])
```

procedimiento Carga(var MM:mat, fil, col: enteros)
Def Carga (MM,fil,col):

```
for i = 0 a fil  
  for j = 0 a col  
    Leer (MM [i,j])
```

programa Principal

FIL=3

COL=4

M = [0] * FIL

for f in range(FIL):

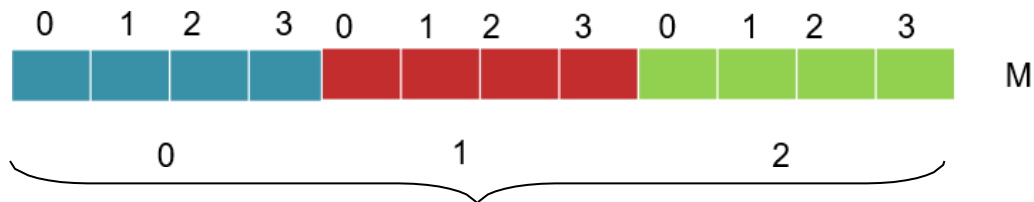
M[f] = [0] * COL

cargar(M,FIL,COL)

mostrar(M,FIL,COL)

Otra manera de declarar la matriz en Python
puede ser: **M= [[0] * COL
 for i in range (FIL)]**

	0	1	2	3
0				
1				
2				



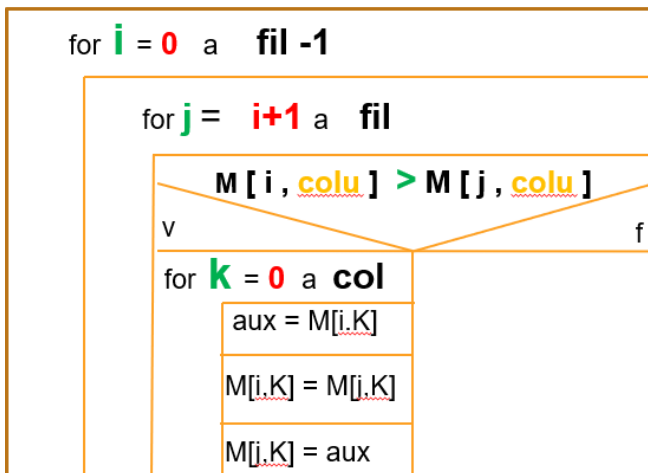
Podemos decir que gráficamente, en Python,
la matriz se puede representar con esta
forma.

ORDENAMIENTO en Bidimensionales

Ordenar un arreglo bidimensional por una columna en particular (Falso Burbuja):

Para ordenar la matriz completa (siempre en función de alguna columna en particular), deberemos intercambiar tantas veces como columnas tenga la matriz, y para ello usaremos otra estructura for. En el siguiente ejemplo podemos ver como pasar como parámetro, por valor, la columna por la que se desea ordenar.

```
procedimiento ORDENO(var M:mat; colu:entero)
Def ORDENO(M.colu):
```



La variable K en este caso se incrementará tantas veces como columnas tenga la matriz.

```
def Ordeno(M,fil,col,COLU):
    for i in range(FIL-1):
        for j in range(i+1,FIL):
            if M[i][COLU] < M[j][COLU]: #ordena de Mayor a Menor
                for k in range (COL):
                    aux=M[i][k]
                    M[i][k]=M[j][k]
                    M[j][k]=aux
```

Notar que en este caso, estamos ordenando en función de una columna pero también podríamos ordenar en función de alguna Fila.

BUSQUEDA SECUENCIAL en Bidimensionales

Programa principal

Carga(M)
Leer (Elem)
Busco(M,Elem)

ejemplo de búsqueda secuencial sobre una matriz de 3 x 3

```
def carga(MM,Tam):
    for i in range (Tam):
        for j in range (0,3):
            MM[i][j]= int(input(' Ingrese nro: '))
```

```
def muestro(MM,Tam):
    print(' ')
    print('.....')
    for i in range (Tam):
        print ('    ', MM[i][0], '    ', MM[i][1], '    ', MM[i][2])
```

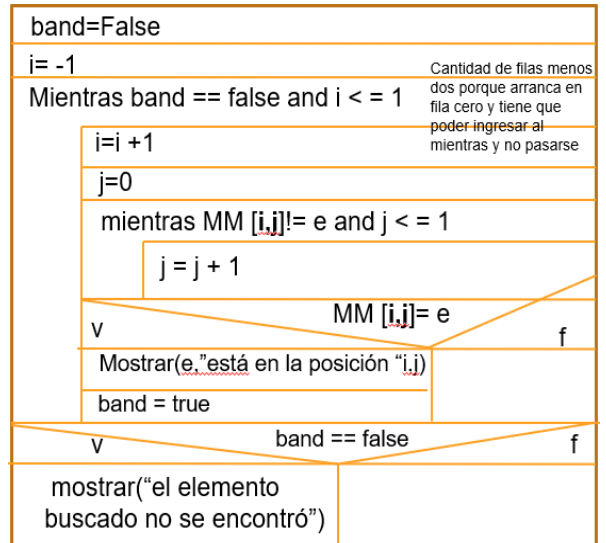
```
def busco(MM,e): #procedimiento
    band = False
    i = -1
    while (band == False and i <=1 ):
        i = i + 1
        j = 0
        while (MM[i][j] != e and j <=1 ):
            j = j + 1
        if MM[i][j] == e:
            print("El número: ",e,"se encontró en la posición: ", i,j)
            band= True
    if band== False:
        print("El elemento:",e," no se encontró")
```

#Programa Principal

```
FIL=3
COL=3
M = [0] * FIL
for f in range(FIL):
    M[f] = [0] * COL
```

```
carga (M,FIL)
muestro(M,FIL)
N = int(input("Elemento a buscar: "))
busco(M,N) #invoco al procedimiento buscar. Notar que i,j son
variables locales de buscar
```

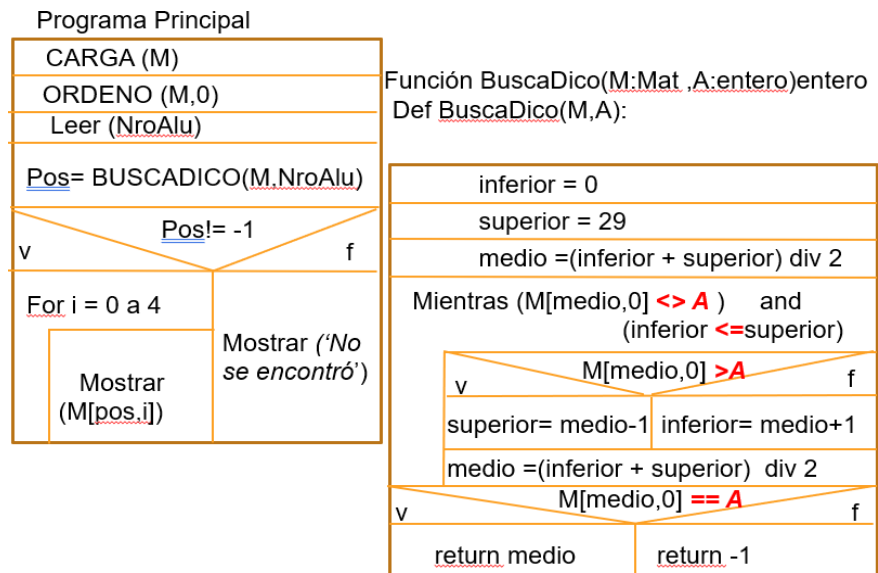
def busco(MM,e):



DICOTOMICA en BIDIMENSIONAL

(suponiendo una matriz de 30 filas x 5 columnas y previamente ordenada por la columna 0)

Usando mientras como estructura de repetición y una función que retorne la posición del medio ó -1 si no lo encuentra.



```
def buscaDico(M,A): # dos return y no uso la variable posición
    inferior = 0
    superior= 29
    medio = (inferior+superior)//2
    while (M [medio][0]!=A) and (inferior<=superior):
        if(A>M[medio][0]):
            inferior= medio + 1
        else:
            superior = medio - 1
            medio = (inferior+superior)//2
    if (M[medio][0] == A):
        return medio
    else:
        return -1
```

Recordar que NO EXISTE EL ALGORITMO ÚNICO y por lo tanto podemos tener variantes de los distintos algoritmos acá vistos y que todos pueden ser correctos siempre que la LÓGICA sea la adecuada.

Completar esta lectura con los videos propuestos por la cátedra.